



Forward Modeling -- Wave propagation

Emmanuel Ikpesu

ei3294

Hildebrand Department of Petroleum and Geosystems Engineering, Cockrell School of Engineering

Q1.2 PINNs Forward

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

import torch
import torch.nn as nn
import torch.optim as optim

from tqdm import tqdm
```

```
In [2]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# Set random seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)
```

Using device: cuda

```
Out[2]: <torch._C.Generator at 0x1eab34ad350>
```

```
In [3]: # Set random seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)

# Define parameters and initial conditions

dx = 0.01
dt = 0.005
x = torch.arange(0, 1 + dx, dx)
t = torch.arange(0, 2.0 + dt, dt)

x_scaled = x
t_scaled = t

dx_scaled = x_scaled[1] - x_scaled[0]
dt_scaled = t_scaled[1] - t_scaled[0]
```

```

x_grid, t_grid = torch.meshgrid(x_scaled, t_scaled, indexing='ij')

# Collocation points
x_colloc = x_grid.float().reshape(-1, 1).to(device)
t_colloc = t_grid.float().reshape(-1, 1).to(device)

c = 1.0
u_x0 = torch.sin(torch.pi * x_scaled) + 0.5 * torch.sin(3 * torch.pi * x_scaled)
du_dtx0 = torch.zeros(len(x_scaled))
u_0t = torch.zeros(len(t_scaled))
u_Lt = torch.zeros(len(t_scaled))

```

```

In [4]: # Finite Difference Solver for 1D Wave Equation
def fd_solver(u_x0, du_dtx0, u_0t, u_Lt, c, dx, dt):
    """Finite Difference Solver for 1D Wave Equation.
    Args:
        u_x0 (np.ndarray): Initial displacement at t=0.
        du_dtx0 (np.ndarray): Initial velocity at t=0.
        u_0t (np.ndarray): Boundary condition at x=0 for all t.
        u_Lt (np.ndarray): Boundary condition at x=L for all t.
        c (float): Wave speed.
        dx (float): Spatial step size.
        dt (float): Time step size.
    Returns:
        np.ndarray: Displacement u(x,t) over space and time.
    """
    n = len(u_x0) - 1
    m = len(u_0t) - 1
    u = np.zeros((m + 1, n + 1))
    u[0, :] = u_x0
    u[:, 0] = u_0t
    u[:, -1] = u_Lt

    for j in range(0, m):
        for i in range(1, n):
            if j == 0:
                u[j + 1, i] = (u[0, i] + du_dtx0[i] * dt
                               + 0.5 * c**2 * (u[0, i+1] - 2 * u[0, i] + u[0, i-1]) * (dt**2) / (dx**2))
            else:
                u[j + 1, i] = (2 * u[j, i] - u[j - 1, i]
                               + c**2 * (u[j, i + 1] - 2 * u[j, i] + u[j, i - 1]) * (dt**2) / (dx**2))

    return u

u_sol_fd = fd_solver(u_x0, du_dtx0, u_0t, u_Lt, c, dx, dt)

```

```

In [5]: # Test CFL condition
CFL = c * dt / dx

print(f"CFL number: {CFL}")
if CFL > 1:
    raise ValueError("CFL condition not met! Reduce dt or increase dx.")

```

CFL number: 0.5

```

In [6]: CFL_scaled = c * dt_scaled / dx_scaled
print(f"CFL number: {CFL_scaled.item():.4f}")
if CFL_scaled > 1:
    raise ValueError("CFL condition not met in scaled coordinates! Reduce dt or increase dx.")

```

CFL number: 0.5000

```

In [7]: # Set random seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)

class SimpleNN(nn.Module):
    """Standard feedforward neural network"""
    def __init__(self, hidden_size=20, n_layers=8):
        super().__init__()

        layers = []

```

```

layers.append(nn.Linear(2, hidden_size)) # Input: (x, t)

# Hidden Layers
for _ in range(n_layers):
    layers.append(nn.Tanh()) # Smooth activation (important!)
    layers.append(nn.Linear(hidden_size, hidden_size))

layers.append(nn.Tanh())
layers.append(nn.Linear(hidden_size, 1)) # Output: displacement u

self.network = nn.Sequential(*layers)

def forward(self, x):
    return self.network(x)

```

```

In [8]: # Set random seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)

def physics_loss(model, x_colloc, t_colloc, c = 1.0):
    """Compute the physics-based loss for the wave equation"""
    # Enable gradients for collocation points
    # Ensure gradients are enabled for input
    x_colloc = x_colloc.clone().detach().to(device).requires_grad_(True)
    t_colloc = t_colloc.clone().detach().to(device).requires_grad_(True)

    # Predict displacement
    u_pred = model(torch.cat([x_colloc, t_colloc], dim=1))

    # First derivative
    dT_dx = torch.autograd.grad(outputs=u_pred,
                                inputs=x_colloc,
                                grad_outputs=torch.ones_like(u_pred),
                                create_graph=True,
                                retain_graph=True)[0]

    dT_dt = torch.autograd.grad(outputs=u_pred,
                                inputs=t_colloc,
                                grad_outputs=torch.ones_like(u_pred),
                                create_graph=True,
                                retain_graph=True)[0]

    # Second derivative
    d2T_dx2 = torch.autograd.grad(outputs=dT_dx,
                                   inputs=x_colloc,
                                   grad_outputs=torch.ones_like(dT_dx),
                                   create_graph=True,
                                   retain_graph=True)[0]

    d2T_dt2 = torch.autograd.grad(outputs=dT_dt,
                                   inputs=t_colloc,
                                   grad_outputs=torch.ones_like(dT_dt),
                                   create_graph=True,
                                   retain_graph=True)[0]

    # Wave equation residual:  $u_{tt} - c^2 * u_{xx} = 0$ 
    residual = d2T_dt2 - (c ** 2) * d2T_dx2

    # Physics Loss: Mean Squared Error of the residuals
    physics_loss_value = torch.mean(residual**2)

    return physics_loss_value

# Test the physics loss function
test_model = SimpleNN().to(device)

test_loss = physics_loss(test_model, x_colloc, t_colloc)

```

```
print(f"Physics loss (untrained model): {test_loss.item():.6f}")
print("This is large since the model hasn't learned the physics yet!")
```

Physics loss (untrained model): 0.000000

This is large since the model hasn't learned the physics yet!

c:\Users\emman\anaconda3\Lib\site-packages\torch\autograd\graph.py:823: UserWarning: Attempting to run cuBLAS, but there was no current CUDA context! Attempting to set the primary context... (Triggered internally at C:\actions-runner_work\pytorch\pytorch\pytorch\aten\src\ATen\cuda\CublasHandlePool.cpp:180.)

```
return Variable._execution_engine.run_backward( # Calls into the C++ engine to run the backward pass
```

```
In [9]: # Set random seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)

n_boundary = len(t_scaled) * 2
n_initial = len(x_scaled)

def boundary_loss(model, n_boundary):
    """Compute the boundary condition loss"""
    t_bc = torch.linspace(0, 2, n_boundary).to(device).reshape(-1, 1)
    u_left = model(torch.cat([torch.ones_like(t_bc).to(device) * 0, t_bc], dim=1))
    u_right = model(torch.cat([torch.ones_like(t_bc).to(device), t_bc], dim=1))
    bc_loss = torch.mean(u_left**2 + u_right**2)
    return bc_loss

def initial_condition_loss(model, u_x0, n_initial):
    """Compute the initial condition loss"""
    # Initial displacement loss
    x_ic = torch.linspace(0, 1, n_initial).to(device).reshape(-1, 1)
    t_ic = torch.zeros_like(x_ic).to(device).reshape(-1, 1).requires_grad_(True)

    u_pred_ic = model(torch.cat([x_ic, t_ic], dim=1))
    ic_loss = torch.mean((u_pred_ic.flatten() - u_x0.to(device))**2)

    # Initial velocity loss
    u_t_i = torch.autograd.grad(outputs=u_pred_ic,
                                inputs=t_ic,
                                grad_outputs=torch.ones_like(u_pred_ic),
                                create_graph=True,
                                retain_graph=True)[0]

    ic_loss += 0.5 * torch.mean(u_t_i.flatten())**2
    return ic_loss

def corner_loss(model):
    """Compute the corner condition loss"""
    corners_x = torch.tensor([0., 0., 1., 1.]).to(device).reshape(-1, 1)
    corners_t = torch.tensor([0., 2., 0., 2.]).to(device).reshape(-1, 1)
    corners = torch.cat([corners_x, corners_t], dim=1)
    u_corners = model(corners)
    corner_loss = torch.mean(u_corners**2)
    return corner_loss

# Test the boundary and initial condition loss function
test_model = SimpleNN().to(device)
bc_loss, ic_loss, c_loss = boundary_loss(test_model, n_boundary), initial_condition_loss(test_model, u_x0, n_in:
print(f"Boundary loss (untrained model): {bc_loss.item():.6f}")
print(f"Initial condition loss (untrained model): {ic_loss.item():.6f}")
print(f"Corner loss (untrained model): {c_loss.item():.6f}")
print("This is large since the model hasn't learned the physics yet!")
```

Boundary loss (untrained model): 0.005823

Initial condition loss (untrained model): 0.699410

Corner loss (untrained model): 0.002922

This is large since the model hasn't learned the physics yet!

```
In [10]: def train_model(model, x_colloc, t_colloc, u_x0, n_boundary, n_initial, epochs=5000, lr=1e-3, c=1.0, p_lambda=1
        , b_lambda=1.0, i_lambda=1.0, co_lambda=1.0):
    """Train the PINN model"""
```

```

optimizer = optim.Adam(model.parameters(), lr=lr)

physics_losses = []
boundary_losses = []
initial_losses = []
corner_losses = []
total_losses = []

for epoch in tqdm(range(epochs)):
    optimizer.zero_grad()

    p_loss = physics_loss(model, x_colloc, t_colloc, c)
    b_loss = boundary_loss(model, n_boundary)
    i_loss = initial_condition_loss(model, u_x0, n_initial)
    c_loss = corner_loss(model)

    total_loss = p_lambda * p_loss + b_lambda * b_loss + i_lambda * i_loss + co_lambda * c_loss

    physics_losses.append(p_loss.item())
    boundary_losses.append(b_loss.item())
    initial_losses.append(i_loss.item())
    corner_losses.append(c_loss.item())
    total_losses.append(total_loss.item())

    total_loss.backward()
    optimizer.step()

    if epoch % 500 == 0:
        print(f"Epoch {epoch}, Total Loss: {total_loss.item():.6f}, Physics Loss: {p_loss.item():.6f}, Boun

return model, physics_losses, boundary_losses, initial_losses, corner_losses, total_losses

```

```

In [11]: # Plot Collocation Points
plt.figure(figsize=(8, 6))

# Plot interior collocation points
plt.scatter(x_colloc.cpu().numpy(), t_colloc.cpu().numpy(), s=1, color='blue', alpha=0.5)

# Plot boundary collocation points
bc_t = torch.linspace(0, 2, n_boundary).to(device).reshape(-1, 1)
x_left = torch.ones_like(bc_t).to(device) * 0
x_right = torch.ones_like(bc_t).to(device)
plt.scatter(x_left.cpu().numpy(), bc_t.cpu().numpy(), s=4, color='green', alpha=0.5, marker='*', label='Boundary')
plt.scatter(x_right.cpu().numpy(), bc_t.cpu().numpy(), s=4, color='green', alpha=0.5, marker='*', label='Bounda')

# Plot initial condition points
ic_x = torch.linspace(0, 1, n_initial).to(device).reshape(-1, 1)
t_initial = torch.zeros_like(ic_x).to(device).reshape(-1, 1)
plt.scatter(ic_x.cpu().numpy(), t_initial.cpu().numpy(), s=4, color='red', alpha=0.5, marker='*', label='Initial')

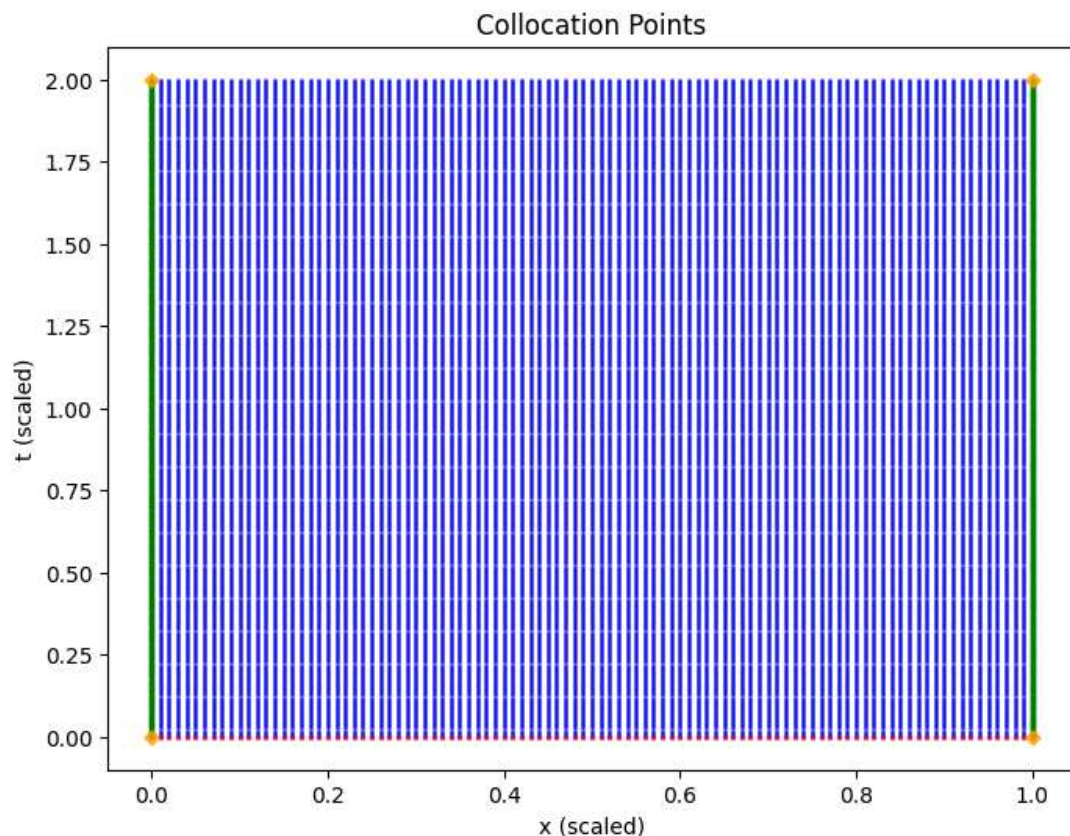
# Plot corner points
corners_x = torch.tensor([0., 0., 1., 1.]).to(device).reshape(-1, 1)
corners_t = torch.tensor([0., 2., 0., 2.]).to(device).reshape(-1, 1)
plt.scatter(corners_x.cpu().numpy(), corners_t.cpu().numpy(), s=20, color='orange', alpha=0.8, marker='D', label='Corners')

plt.xlabel('x (scaled)')
plt.ylabel('t (scaled)')

plt.title('Collocation Points')

```

Out[11]: Text(0.5, 1.0, 'Collocation Points')



In [12]: *# Set random seeds for reproducibility*

```
np.random.seed(42)
torch.manual_seed(42)

model = SimpleNN().to(device)

trained_model, physics_losses, boundary_losses, initial_losses, corner_losses, total_losses = train_model(
    model, x_colloc, t_colloc, u_x0, n_boundary, n_initial, epochs=15000, lr=1e-3,
    p_lambda=1, b_lambda=5.0, i_lambda=5.0, co_lambda=5.0)
```

0% | 6/15000 [00:00<11:28, 21.79it/s]

Epoch 0, Total Loss: 3.540776, Physics Loss: 0.000000, Boundary Loss: 0.005823, Initial Loss: 0.699410, Corner Loss: 0.002922

3% | 508/15000 [00:10<04:51, 49.80it/s]

Epoch 500, Total Loss: 0.784748, Physics Loss: 0.037587, Boundary Loss: 0.034757, Initial Loss: 0.107475, Corner Loss: 0.007200

7% | 1005/15000 [00:20<04:47, 48.67it/s]

Epoch 1000, Total Loss: 0.707536, Physics Loss: 0.029942, Boundary Loss: 0.028797, Initial Loss: 0.095833, Corner Loss: 0.010889

10% | 1511/15000 [00:31<04:32, 49.55it/s]

Epoch 1500, Total Loss: 0.672294, Physics Loss: 0.031560, Boundary Loss: 0.024128, Initial Loss: 0.087581, Corner Loss: 0.016438

13% | 2005/15000 [00:41<04:29, 48.20it/s]

Epoch 2000, Total Loss: 0.689099, Physics Loss: 0.022337, Boundary Loss: 0.027486, Initial Loss: 0.101325, Corner Loss: 0.004541

17% | 2509/15000 [00:51<04:08, 50.36it/s]

Epoch 2500, Total Loss: 0.541481, Physics Loss: 0.015925, Boundary Loss: 0.007098, Initial Loss: 0.092314, Corner Loss: 0.005699

20% | 3008/15000 [01:01<03:56, 50.73it/s]

Epoch 3000, Total Loss: 0.782966, Physics Loss: 0.027963, Boundary Loss: 0.036758, Initial Loss: 0.084323, Corner Loss: 0.029920

23% | 3511/15000 [01:11<03:54, 48.97it/s]

Epoch 3500, Total Loss: 0.562109, Physics Loss: 0.014747, Boundary Loss: 0.011117, Initial Loss: 0.082780, Corner Loss: 0.015575

27% | 4005/15000 [01:21<03:45, 48.69it/s]

Epoch 4000, Total Loss: 0.497362, Physics Loss: 0.012572, Boundary Loss: 0.003500, Initial Loss: 0.087007, Corner Loss: 0.006451

30% | ████████ | 4507/15000 [01:32<03:38, 48.02it/s]

Epoch 4500, Total Loss: 0.546470, Physics Loss: 0.015972, Boundary Loss: 0.010804, Initial Loss: 0.081498, Corner Loss: 0.013798

33% | ████████ | 5007/15000 [01:42<03:20, 49.79it/s]

Epoch 5000, Total Loss: 0.482022, Physics Loss: 0.013962, Boundary Loss: 0.003254, Initial Loss: 0.084645, Corner Loss: 0.005713

37% | ████████ | 5508/15000 [01:52<03:06, 50.99it/s]

Epoch 5500, Total Loss: 0.470504, Physics Loss: 0.015835, Boundary Loss: 0.003469, Initial Loss: 0.081771, Corner Loss: 0.005693

40% | ████████ | 6010/15000 [02:02<03:01, 49.63it/s]

Epoch 6000, Total Loss: 0.452210, Physics Loss: 0.019517, Boundary Loss: 0.003952, Initial Loss: 0.076767, Corner Loss: 0.005819

43% | ████████ | 6507/15000 [02:12<02:53, 49.00it/s]

Epoch 6500, Total Loss: 0.427048, Physics Loss: 0.022195, Boundary Loss: 0.004496, Initial Loss: 0.071174, Corner Loss: 0.005300

47% | ████████ | 7005/15000 [02:23<02:43, 48.80it/s]

Epoch 7000, Total Loss: 0.555801, Physics Loss: 0.056381, Boundary Loss: 0.013903, Initial Loss: 0.062041, Corner Loss: 0.023940

50% | ████████ | 7510/15000 [02:33<02:28, 50.36it/s]

Epoch 7500, Total Loss: 0.370535, Physics Loss: 0.025194, Boundary Loss: 0.007608, Initial Loss: 0.052528, Corner Loss: 0.008932

53% | ████████ | 8005/15000 [02:43<02:20, 49.93it/s]

Epoch 8000, Total Loss: 0.316138, Physics Loss: 0.022919, Boundary Loss: 0.008349, Initial Loss: 0.046102, Corner Loss: 0.004192

57% | ████████ | 8508/15000 [02:53<02:13, 48.65it/s]

Epoch 8500, Total Loss: 0.270891, Physics Loss: 0.022502, Boundary Loss: 0.009428, Initial Loss: 0.037379, Corner Loss: 0.002871

60% | ████████ | 9008/15000 [03:03<02:01, 49.25it/s]

Epoch 9000, Total Loss: 0.198737, Physics Loss: 0.027312, Boundary Loss: 0.011378, Initial Loss: 0.020851, Corner Loss: 0.002056

63% | ████████ | 9507/15000 [03:14<01:51, 49.36it/s]

Epoch 9500, Total Loss: 0.160770, Physics Loss: 0.026700, Boundary Loss: 0.012170, Initial Loss: 0.013936, Corner Loss: 0.000708

67% | ████████ | 10010/15000 [03:24<01:42, 48.56it/s]

Epoch 10000, Total Loss: 0.141343, Physics Loss: 0.020790, Boundary Loss: 0.012488, Initial Loss: 0.010948, Corner Loss: 0.000675

70% | ████████ | 10511/15000 [03:34<01:33, 47.87it/s]

Epoch 10500, Total Loss: 0.129908, Physics Loss: 0.017036, Boundary Loss: 0.012739, Initial Loss: 0.009333, Corner Loss: 0.000502

73% | ████████ | 11010/15000 [03:44<01:24, 47.34it/s]

Epoch 11000, Total Loss: 0.124654, Physics Loss: 0.013832, Boundary Loss: 0.012305, Initial Loss: 0.009269, Corner Loss: 0.000591

77% | ████████ | 11505/15000 [03:55<01:13, 47.26it/s]

Epoch 11500, Total Loss: 0.119489, Physics Loss: 0.012393, Boundary Loss: 0.012100, Initial Loss: 0.008837, Corner Loss: 0.000482

80% | ████████ | 12009/15000 [04:05<01:01, 48.97it/s]

Epoch 12000, Total Loss: 0.111419, Physics Loss: 0.011610, Boundary Loss: 0.012758, Initial Loss: 0.006897, Corner Loss: 0.000306

83% | ████████ | 12507/15000 [04:15<00:51, 48.72it/s]

Epoch 12500, Total Loss: 0.107705, Physics Loss: 0.010634, Boundary Loss: 0.012689, Initial Loss: 0.006480, Corner Loss: 0.000246

87% | ████████ | 13008/15000 [04:25<00:40, 49.01it/s]

Epoch 13000, Total Loss: 0.107046, Physics Loss: 0.009243, Boundary Loss: 0.012124, Initial Loss: 0.007280, Corner Loss: 0.000157

90% | ████████ | 13505/15000 [04:36<00:31, 46.82it/s]

Epoch 13500, Total Loss: 0.121857, Physics Loss: 0.022308, Boundary Loss: 0.012746, Initial Loss: 0.006777, Corner Loss: 0.000387

93% | ████████ | 14008/15000 [04:46<00:20, 48.05it/s]

Epoch 14000, Total Loss: 0.100316, Physics Loss: 0.008099, Boundary Loss: 0.012009, Initial Loss: 0.006232, Corner Loss: 0.000202

97% | ████████ | 14506/15000 [04:57<00:10, 47.11it/s]

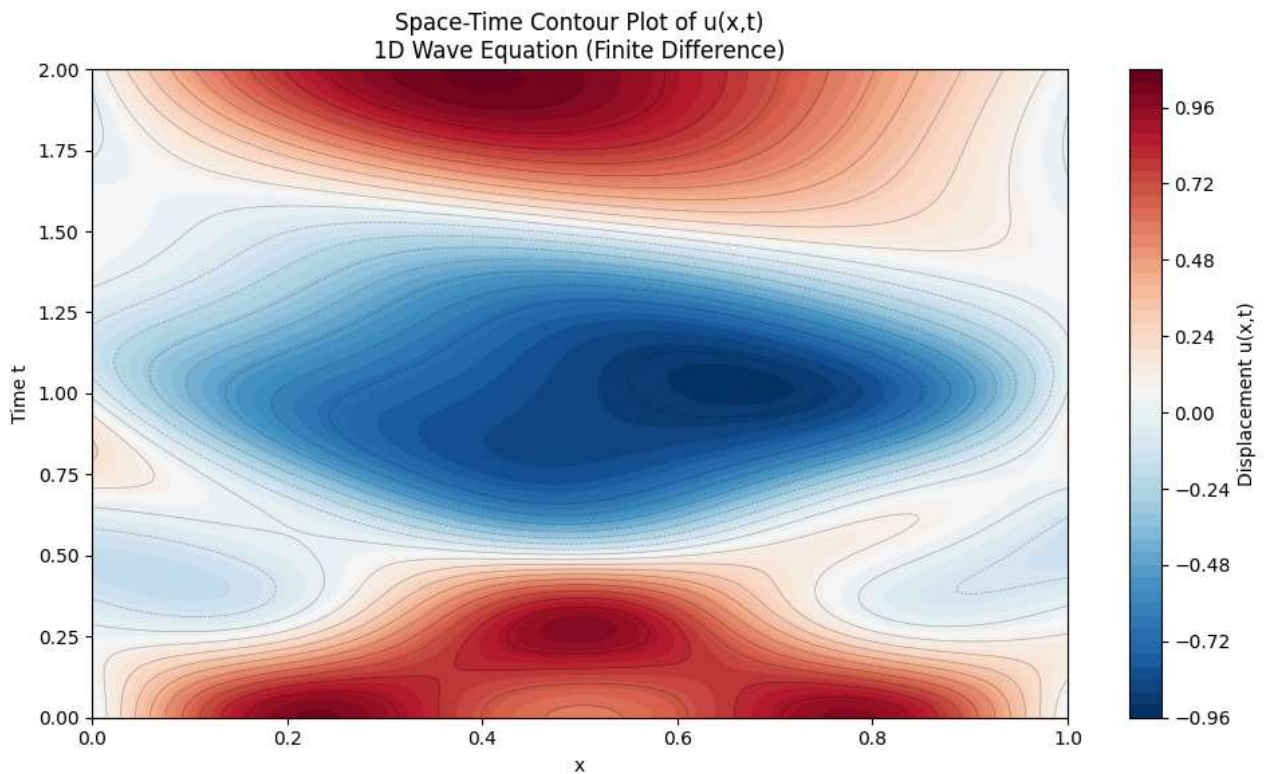
Epoch 14500, Total Loss: 0.098564, Physics Loss: 0.008685, Boundary Loss: 0.012182, Initial Loss: 0.005577, Corner Loss: 0.000217

100%|██████████ 15000/15000 [05:07<00:00, 48.78it/s]

```
In [13]: # Set random seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)

# Make predictions
with torch.no_grad():
    u_pred_flat = trained_model(torch.cat([x_colloc, t_colloc], dim=1))
    u_pred = u_pred_flat.reshape(x.shape[0], t.shape[0]).cpu().numpy()
```

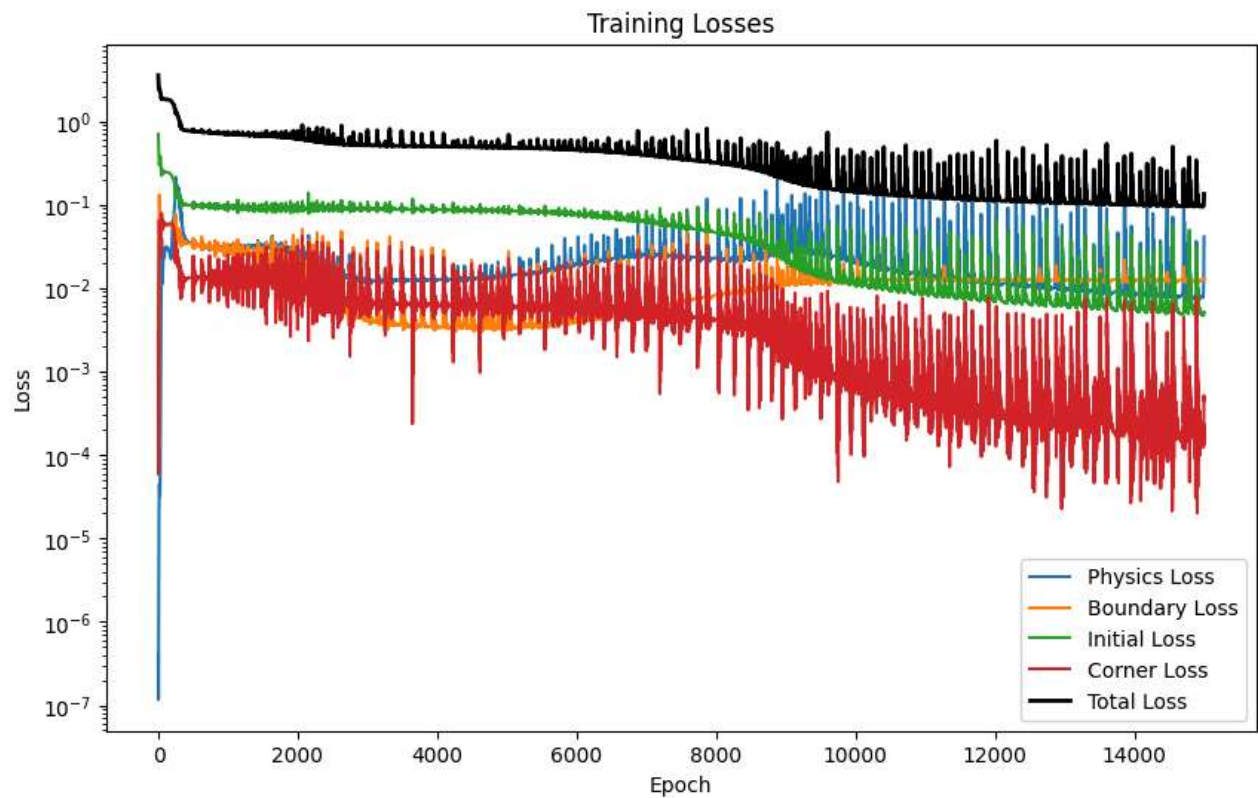
```
In [14]: # Space-time contour plot
X, T_grid = np.meshgrid(x, t, indexing='ij')
plt.figure(figsize=(10, 6))
contour = plt.contourf(X, T_grid, u_pred, levels=50, cmap='RdBu_r')
plt.colorbar(contour, label='Displacement u(x,t)')
plt.contour(X, T_grid, u_pred, levels=20, colors='black', alpha=0.3, linewidths=0.5)
plt.xlabel('x')
plt.ylabel('Time t')
plt.title('Space-Time Contour Plot of u(x,t)\n1D Wave Equation (Finite Difference)')
plt.tight_layout()
plt.show()
```



In []:

```
In [15]: # Plot training losses
plt.figure(figsize=(10, 6))
plt.semilogy(physics_losses, label='Physics Loss')
plt.semilogy(boundary_losses, label='Boundary Loss')
plt.semilogy(initial_losses, label='Initial Loss')
plt.semilogy(corner_losses, label='Corner Loss')
plt.semilogy(total_losses, label='Total Loss', linewidth=2, color='black')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Training Losses')
```

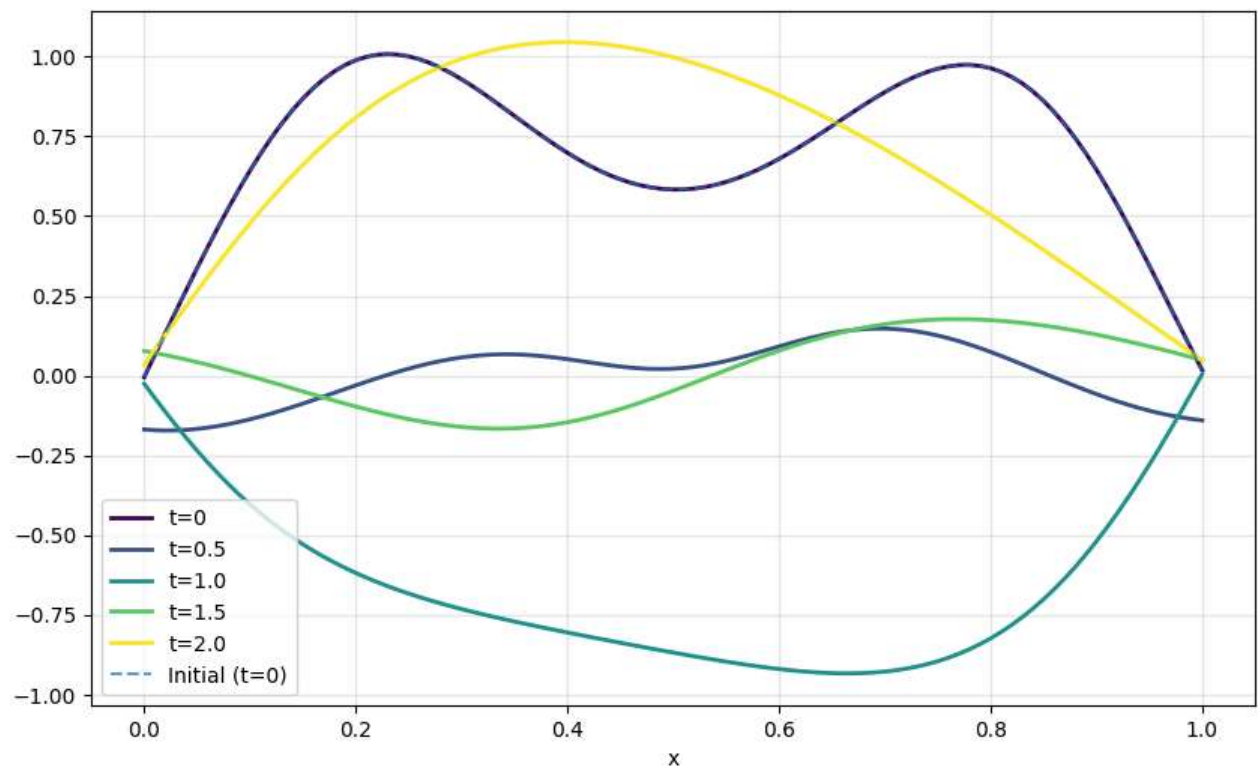
Out[15]: Text(0.5, 1.0, 'Training Losses')



```
In [27]: t_eval = [0,0.5,1.0,1.5,2.0]
t_eval_idx = [list(t).index(v) for v in t_eval]
colors = plt.cm.viridis(np.linspace(0, 1, len(t_eval)))

plt.figure(figsize=(10, 6))
for idx, time in zip(t_eval_idx, t_eval):
    plt.plot(x, u_pred[:, idx].T, label=f't={time}', linewidth=2, color=colors[t_eval_idx.index(idx)])
plt.plot(x, u_pred[:, 0].T, label=f'Initial (t=0)', linestyle='--', linewidth=1.5, alpha=0.7)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlabel('x')
```

Out[27]: Text(0.5, 0, 'x')



iv) Comparison with FD solution (absolute error plot) at 2.0 s

```
In [23]: plt.plot(x, u_sol_fd[-1, :], label=f't=FD solution at 2.0s', linewidth=2)
plt.plot(x, u_pred[:, -1], label=f't=PINNs solution at 2.0s', linewidth=2)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlabel('x')
plt.ylabel('Displacement')
plt.title('Displacement at t=2.0s')
```

```
Out[23]: Text(0.5, 1.0, 'Displacement at t=2.0s')
```

